

Synthetic Drone Swarm RF Dataset

Team Plan & Implementation Guide • Based on the DroneDetect Dataset • Team of 3

The big idea. DroneDetect only records **one drone at a time**. We will *construct* a fake swarm by:

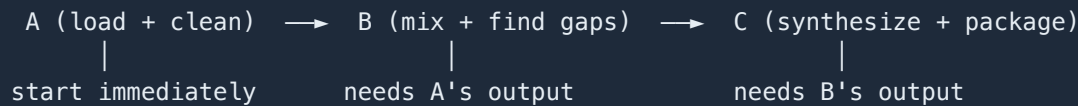
1. Taking two single-drone recordings,
2. Making them take turns on the same channel (strict TDMA),
3. Filling the silent gaps with synthetic "drone-to-drone" messages,
4. Cutting the result into labeled training examples.

⚠ **This is a simulated swarm, not a real capture. This must be stated clearly in the documentation.**

Part 1 — Who Does What (3 People)

Member	Job	Delivers
A — Loader	Read the raw files, clean them, and mark when each drone is "talking" vs "silent".	Clean IQ signal + an on/off activity mask + metadata, for each drone.
B — Mixer	Make the two drones take turns, add them into one stream, and find the silent gaps.	One combined swarm stream + a list of gaps + the slot schedule.
C — Synth & Packaging	Create fake inter-drone signals, insert them into gaps, then cut & label everything.	The final labeled dataset + a data card + a validation report.

Part 2 — Order & Dependencies



- **Hard order:** A → B → C. B needs A's masks; C needs B's combined stream and gaps.
- **Work in parallel from day 1:** agree on the data format (Part 4) first. Then B can test its logic on hand-made fake bursts and C on random noise + fake gaps, while A works on the real files.
- **Checkpoints:** (1) format frozen → (2) A delivers one real drone pair → (3) B delivers combined stream + gaps → (4) full run & review together.

Part 3 — Decide These First (as a Team)

1. **Which two drones?** Pick two *different* drone models, using the "clean" (no-interference) recordings. Different models make the labels meaningful; clean recordings make activity detection easy.
2. **Confirm the file format from the dataset README.** Verify sample rate, center frequency, and number type (int16 vs float32). DroneDetect was captured on a BladeRF (commonly cited ~60 MS/s at ~2.44 GHz), stored as interleaved IQ. **Getting this wrong silently corrupts everything downstream.**
3. **Save raw IQ** (not images). You can always generate spectrograms later; raw IQ is the ground truth and keeps you flexible.

Part 4 — Data Format Contract (freeze this first)

Everyone codes to these shapes. Internally use **samples**; in metadata use **seconds**.

A → B (per drone):

```
iq      : complex array      # I + jQ, normalized
mask    : boolean array      # True where the drone is transmitting
meta    : { fs_hz, fc_hz, drone, condition, source_file }
```

B → C:

```
combined_iq : complex array
slot_map     : list of { start_sample, end_sample, owner: 'D1'|'D2'|'idle' }
gap_list     : list of { start_sample, end_sample, duration_s }
meta         : { fs_hz, fc_hz, tdma_slot_s, drones:[...] }
```

Part 5 — Step-by-Step Guide

STAGE A — Load & Clean (Member A)

1. **Read the file** into complex numbers ($I + jQ$). If int16 interleaved, reshape to (N,2), form `I + 1j*Q`, divide by 32768.
✓ **Check:** plot a spectrogram — you should see clear bursty drone signals. Garbage = wrong format. This 2-minute check saves days.
2. **Match settings:** ensure both drones share the same sample rate & center frequency. If not, resample / frequency-shift. (Usually already the same — but verify.)
3. **Normalize volume** so one drone isn't dominant just because its recording was louder. Record the scale factor.
4. **Detect activity:** slide a window, measure power, threshold it → on/off `mask`. Smooth it so bursts are contiguous. *Start with a fixed threshold; upgrade to two-threshold hysteresis later for cleaner edges.* This mask is Stage A's main product.

STAGE B — Mix into a Fake Swarm (Member B)

5. **Make time slots:** slot 1 → Drone 1, slot 2 → Drone 2, repeating. Set slot length $\approx$ one natural burst (from Stage A). Make it a config parameter.
6. **Gate:** keep each drone's signal only inside its own slots (zero elsewhere). Apply a short raised-cosine taper at slot edges to avoid spectral artifacts.
7. **Add them:** `combined = drone1_gated + drone2_gated`. Since they never overlap, the sum is clean.
✓ **Check:** spectrogram shows the drones alternating in time, never stacked at the same instant.
8. **Find gaps:** mark where *nobody* is transmitting → sorted `gap_list` of (start, end, duration), filtering out gaps too short to be useful. These are where Member C works.

STAGE C — Synthesize & Package (Member C)

9. **Make fake inter-drone packets:** short modulated bursts (e.g. QPSK/OQPSK with root-raised-cosine shaping). Make them *distinguishable* from the drone links so a model has something learnable.
10. **Insert into SOME gaps** (not all — you need "present" and "absent" examples). Scale to a target SNR and add noise so they aren't unrealistically perfect. **Log every insertion** — this log is your ground truth. Use a fixed random seed.
11. **(Optional) Add realism:** a faint background noise floor so silence isn't perfectly empty. Keep it as a toggle for "easy" vs "hard" dataset variants.
12. **Cut & label:** slice into fixed-length examples. For each, record which drones are active and whether an inter-drone message is present (plus optional timing). Save as:

```
dataset/  
  iq/example_00001.npy ...  
  labels.csv           # id, drones_active, interdrone_present, snr, ...  
  metadata.json        # fs, fc, tdma config, source drones, seed  
  DATA_CARD.md        # how it was built + "this is simulated"
```

13. **Validate:** plot spectrograms, check labels are balanced, train a quick baseline classifier. If it hits ~100% instantly → fake signals are too obvious (a leak); if ~random → something is mislabeled.

Part 6 — Top Things That Will Bite You

#	Risk	Practical Solution
1	Wrong file format → silent corruption	Verify from README; confirm with a spectrogram before anything else.
2	Different sample rates / center freq	Resample & frequency-shift before mixing; assert they match in code.
3	Bad activity detection poisons gaps & labels	Start with a fixed threshold, upgrade to hysteresis; check visually vs spectrograms.
4	Fake signals too clean → model cheats	Add AWGN + random power/SNR sweep; verify baseline isn't trivially perfect.
5	Hard slot cuts create edge artifacts	Apply short raised-cosine tapers at slot boundaries.
6	Not reproducible	Fix random seeds; log every insertion; save full config in metadata.
7	Huge files at ~60 MS/s	Process in chunks / memmap; decimate early if the signal is narrowband.
8	"Strict TDMA" is a simplification	Document it as a modeling choice; consider an FDMA/overlap variant later.

Part 7 — Suggested Tools

- **Python + NumPy/SciPy** — IQ handling, filtering, resampling.
- **Matplotlib** — PSD & spectrogram validation.
- **HDF5 (h5py) or .npy** for signals; **CSV/Parquet** for labels.
- Optional: **commpy / GNU Radio** for realistic modulation; **scikit-learn / PyTorch** for the baseline validation classifier.
- One **Git repo**, three modules (A / B / C) matching the Part 4 contract.

Synthetic Drone Swarm RF Dataset — Team Plan • DroneDetect-based • Remember: this dataset is a simulated swarm and must be documented as such.